

ADR-02 — Implementación del patrón DTO para desacoplar entidades de la API REST

Estado

Propuesto

Contexto

Durante el análisis del backend del sistema **ERP Iglesias** se observó que algunos controladores REST retornan directamente **entidades JPA** como respuesta de los endpoints.

El proyecto utiliza **Spring Boot**, **Spring Data JPA** y **Repository Pattern**, lo cual facilita el acceso a datos mediante repositorios.

Sin embargo, actualmente las **entidades de base de datos se están utilizando también como modelo de respuesta en la API**, lo cual genera problemas de diseño arquitectónico.

Esto implica que la estructura interna del modelo de persistencia se expone directamente al cliente.

Situación actual:

```
Controller
↓
Repository
↓
Entity (respuesta HTTP)
```

En este enfoque la entidad cumple **dos responsabilidades al mismo tiempo**:

- modelo de persistencia
- modelo de exposición en la API

Esto genera varios problemas:

- alto acoplamiento entre la capa web y la capa de persistencia
 - exposición de la estructura interna de la base de datos
 - dificultad para evolucionar el modelo de dominio
 - riesgo de exponer campos sensibles
-

Problema

Cuando las entidades JPA se retornan directamente desde los controladores:

- cualquier cambio en la entidad puede romper el contrato de la API
- se exponen atributos que no deberían ser públicos
- se dificulta el control sobre la estructura de las respuestas HTTP

Por ejemplo, un controlador podría retornar directamente una entidad:

```
@GetMapping("/persons/{id}")
public Person getPerson(@PathVariable Long id) {
    return personRepository.findById(id).orElseThrow();
}
```

En este caso:

- la entidad `Person` es enviada directamente al cliente
- la API depende completamente del modelo de base de datos

Esto viola el **principio de responsabilidad única (SRP)**.

Decisión Arquitectónica

Se propone implementar el **patrón DTO (Data Transfer Object)** para separar el modelo de persistencia del modelo expuesto en la API REST.

Los controladores dejarán de retornar entidades y utilizarán **DTOs como objetos de respuesta**.

La arquitectura quedará organizada de la siguiente forma:

```
Controller
↓
Service
↓
Repository
↓
Entity
↓
Mapper
↓
DTO
```

Las entidades permanecerán únicamente en la **capa de persistencia**, mientras que los DTO serán utilizados para comunicación con el cliente.

Patrón de Diseño Aplicado

Data Transfer Object (DTO)

Este patrón permite transportar datos entre capas del sistema sin exponer directamente las entidades del dominio.

Principio SOLID Aplicado

Single Responsibility Principle (SRP)

Cada componente tendrá una única responsabilidad:

Controller → manejar endpoints HTTP Service → lógica de negocio Repository → acceso a datos DTO → transporte de datos entre capas

Cambio Concreto Propuesto

Se propone crear **DTOs y Mappers** para las entidades principales del sistema.

Ejemplo de DTOs:

```
PersonDTO
CourseDTO
EnrollmentDTO
PaymentDTO
```

Los controladores utilizarán DTOs en lugar de entidades.

Código Actual (ANTES)

Ejemplo simplificado de un controlador actual:

```
@GetMapping("/persons/{id}")
public Person getPerson(@PathVariable Long id) {

    Person person = personRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Persona no encontrada"));

    return person;
}
```

Arquitectura actual:

```
Controller
↓
Repository
↓
Database
```

Problema:

- la entidad `Person` se expone directamente en la API.

Mejora Propuesta (DESPUÉS)

El controlador retorna un **DTO**.

```
@GetMapping("/persons/{id}")
public PersonDTO getPerson(@PathVariable Long id) {

    Person person = personService.findById(id);

    return PersonMapper.toDTO(person);
}
```

DTO:

```
public class PersonDTO {

    private Long id;
    private String firstName;
    private String lastName;

}
```

Mapper:

```
public class PersonMapper {

    public static PersonDTO toDTO(Person person) {

        PersonDTO dto = new PersonDTO();

        dto.setId(person.getId());
        dto.setFirstName(person.getFirstName());
        dto.setLastName(person.getLastName());

    }

}
```

```
        return dto;
    }
}
```

Arquitectura resultante:

```
Controller
↓
Service
↓
Repository
↓
Entity
↓
Mapper
↓
DTO
```

Beneficios Arquitectónicos

La implementación del patrón DTO proporciona:

- desacoplamiento entre la API y el modelo de base de datos
- mayor control sobre los datos expuestos
- reducción del acoplamiento entre capas
- mayor mantenibilidad del sistema
- mejor organización del código
- posibilidad de evolucionar el modelo interno sin romper la API

Impacto en el Sistema

El cambio no afecta el comportamiento funcional del sistema.

Los endpoints REST continúan funcionando de la misma forma, pero ahora retornan **DTOs en lugar de entidades**.

Esto permite mejorar la arquitectura del sistema sin modificar la interfaz pública de la API.

Resultado Arquitectónico

Arquitectura final propuesta:

```
Controller
↓
Service
↓
Repository
↓
Entity
↓
Mapper
↓
DTO
```

Esta estructura mejora la organización del sistema, facilita su evolución y permite aplicar buenas prácticas de diseño de software.